PA2 - ACTIVIDAD 4: LISTAS DOBLES Y CIRCULARES

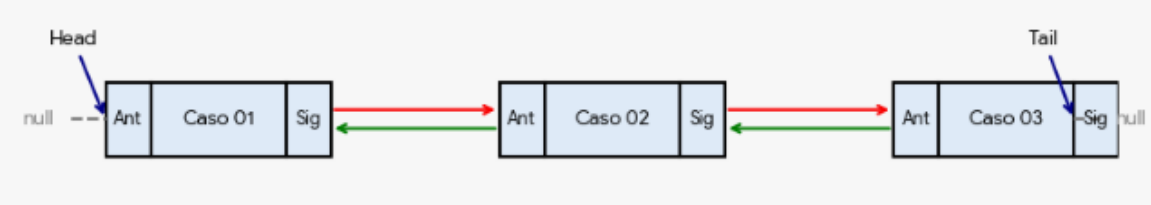
Situación con Lista Doblemente Enlazada (Módulo de Auditoría y Navegación)

• **Planteamiento de la situación:**

El sistema requiere un Módulo de Mesa de Control y Supervisor de Operaciones donde un analista visualiza una solicitud de atención en tiempo real. El analista debe auditar los antecedentes inmediatos de la orden actual (retroceder) o evaluar el caso que sigue en la cola (avanzar) para validar datos de reubicación de manera fluida.

• **Justificación técnica:**

Una lista simplemente enlazada solo permite avanzar hacia adelante (nodo.siguiente). Si se necesita regresar al nodo anterior, el sistema se ve obligado a reiniciar el recorrido de lectura desde la cabeza (head), generando un costo computacional de O(n). Al implementar una lista doblemente enlazada, cada nodo almacena un puntero al elemento siguiente y otro al anterior. Esto permite la navegación bidireccional inmediata con un costo constante de O(1), agilizando el reajuste dinámico de enlaces al mover o eliminar registros en cualquier punto del flujo.



Situación con Lista Enlazada Circular (Auditoría Cíclica Perpetua)

• **Planteamiento de la situación:**

Se requiere implementar una Cola de Distribución Automática / Cíclica de Casos Pendientes (Algoritmo de asignación Round Robin). El sistema debe evaluar constantemente el estado de las solicitudes asignadas a los operadores de soporte; cuando el motor llega al último elemento de la cola de atención, debe reiniciar la inspección desde el primer elemento de manera inmediata sin detener el hilo.

• **Justificación técnica**:

En las listas convencionales, el último nodo contiene un puntero final con referencia vacía (null). Esto obligaría a los algoritmos del backend a evaluar condicionales constantes de fin de estructura para resetear manualmente el puntero hacia la cabeza (actual = head). La lista circular simple optimiza este proceso al enlazar directamente el campo siguiente del último nodo hacia la dirección de memoria de la cabeza (head). Esto asegura una continuidad infinita libre de condicionales de quiebre en tiempo de ejecución, optimizando los ciclos recurrentes del despachador de solicitudes.

